Art analysis of the classification from 2024

This has been written in 2025.

The main objective is to classify the publications by themes and sub-themes.

The work from 2024

It seems to work well. However there are two possible issues:

- There are not enough themes to cover all the possible scientific publications.

Let's consider only the publications that are related to the given themes.

- The classification is only made on the *abstract* of the publication, and not on the title or even the keywords that the *HAL* **API** could give for istance.

I think it could be good if we gave the title, the abstract and the retrieved keywords altogether to the *classify_abstract_combined()* function.

It will give the related *theme* if the publication is within the covered range of themes.

Development server using Flask

In fact, it may be necessary to replace the *Flask API* because:

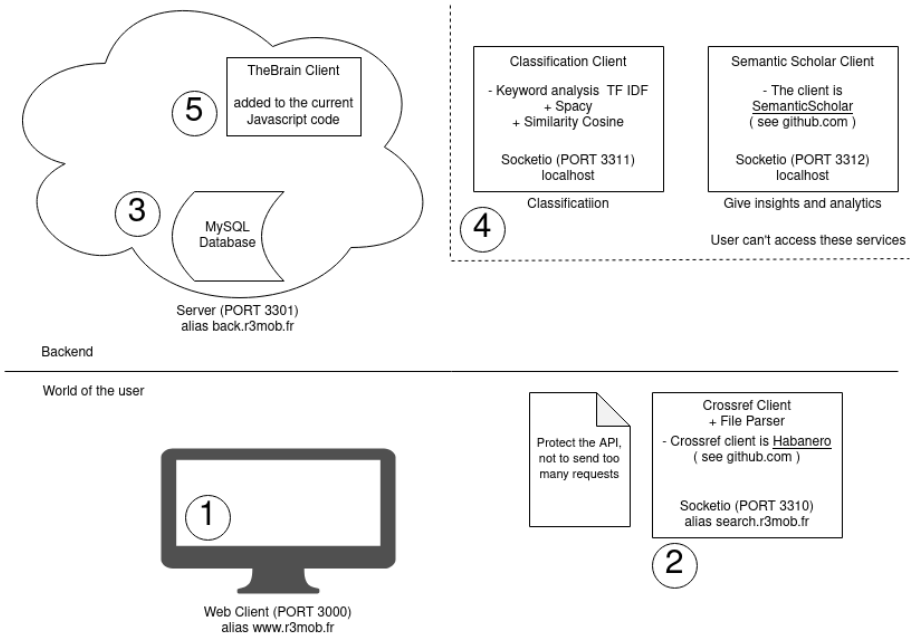
```
WARNING: This is a development server.
Do not use it in a production deployment.
Use a production WSGI server instead.
```

So anyway, the current *Flask API* will have to be reworked. Here are some ideas:

1. **Use HTTP requests with *Flask***. This is the easiest way because it requires small changes, however this is not designed for that I suppose. In fact, the only good way to get a *Flask* server for production is to use ***gunicorn***.
2. **Use *Child Processes* in Node.js**. Here is a more detailed guide ***right here***.
3. **Use *Websocket* like *Socketio***. I don't want the *websocket* to require an authentication because it will be too hard to code. That's why the client must not have the right to communicate with the *websocket*. Only the server should do that. So for this method, the server classifies the publication and not the client.

Why is it cool to use `flask_socketio` for production? Because the function `flask_socketio.SocketIO.run()` directly calls **python-gevent** or **python-eventlet** if it is available, ***the doc*** says.

For any of these methods, here is the generic plan on this picture:



1. The user is connecting.
2. The user is importing a publication.
3. The server receives the new publication and stores its metadata.
4. The server retrieves analytics and insights and then classify it.
5. The admin chooses whether the publication will go in the R3MOB brain, using the Admin page on the web client.

Figure 1: Classification Module

My own thoughts about the idea

The solution from 2023 is good, nothing to add, except that the model was not trained, because of the lack of labellised data.

For instance, for the use of *TF-IDF*, this is the former classifying function:

```
def classify_abstract_TF_IDF(abstract_text, themes_keywords):
    abstract_text = ' '.join(preprocess_text(abstract_text))
    themes_keywords = expand_and_preprocess_keywords(themes_keywords)
```

```
    vectorizer = TfidfVectorizer()
    abstract_tfidf = vectorizer.fit_transform([abstract_text])
    abstract_themes = []
    for theme, keywords in themes_keywords.items():
        theme_tfidf = vectorizer.transform(keywords)
        similarity = (abstract_tfidf * theme_tfidf.T).toarray()

        # Here is a "max" not to reach.
        if similarity.max() > 0.15:
            abstract_themes.append(theme)
        # </max not to reach>

    return abstract_themes
```

You can see here that the *corpus* given to `vectorizer` is composed of just one element, called `abstract_text`.

This is a misunderstanding of how *TF-IDF* works. Indeed, `vectorizer` is waiting for a *corpus*, which is a set of `labellized abstracts`.

Furthermore, The `similarity.max()` is not a good way to classify results. It is better to use a classifier.

This will be discussed further.

The different solutions that already exist.

It is possible to use a *Natural Language Processing* to get good results without having to train a model. However, the precision will be cap to the max precision of the model, which is usually something like *70% of precision*.

Edit from June 23, 2025: Yes, there are enough data

It is possible to retrieve more than one abstract for a given publication. Indeed, thanks to *Crossref*, it is possible to recursively retrieve publications from the references, or even from the citing publications.

In the current *keyword* based model, the goal is to get a lot of words in the text to classify. **To concatenate abstracts from the publication, from the references, titles and container-titles from the references** will for sure enhance the classification result. The issue of "not having enough data to classify" is not a matter now!

Edit from June 23, 2025: What to do when the abstract/title is not written in english?

It is not possible to ask *Crossref* to translate the text before sending it (unfortunately).

I suppose that this issue is related to the current model based on *keywords*. In fact, a *NLP* like *BERT* will not care about the language. However, the keywords are from the *spacy* module called `en_core_web_lg`.

It could be possible to install various modules from *spacy*, one for each language for instance. This will take a lot of memory. This solution could also alterate the *time to classify* a text. Furthermore, there will be too many keywords, the classification module will flood.

So, my idea is to **temporary** use a *llm*, using an *api key* that will translate the text, until the *TF IDF* method is implemented.

EOF

Edit from July 03, 2025: a *state of art* on the classification

Given a publication, or a research paper, The objective is to find a *theme*, *scientific themes*, *axes* and *mobility types* related to it such as:

- **Themes**:

```
{
  "Communications": [
    "Communication networks",
    "Connectivity",
    "Signal transmission",
    "Telecommunications",
    "Information exchange",
    "Wireless communication",
    "Broadcasting",
    "Digital communication",
    "Internet",
    "5G",
    "Data transfer",
    "V2X communication",
    "Satellite communication",
    "Latency",
    "Network coverage"
  ],
  "Energie": [
    "Energy production",
    "Renewable energy",
    "Energy consumption",
    "Power grid",
    "Electricity",
    "Energy transition",
```

```
    "Fossil fuels",
    "Hydrogen",
    "Battery storage",
    "Energy efficiency",
    "Solar power",
    "Wind energy",
    "Decarbonization",
    "Energy infrastructure",
    "Smart grid"
  ],
  "Logistique": [
    "Logistics",
    "Supply chain",
    "Freight transport",
    "Warehouse",
    "Distribution",
    "Last mile delivery",
    "Intermodality",
    "Multimodal transport",
    "Inventory management",
    "Logistics optimization",
    "Transport hubs",
    "Fleet management",
    "Cold chain",
    "Goods transportation",
    "Logistics network"
  ],
  "Matériaux": [
    "Materials",
    "Material science",
    "Composite materials",
    "Lightweight structures",
    "Recyclable materials",
    "Mechanical properties",
    "Durability",
    "Resistance",
    "Nanomaterials",
    "Thermal properties",
    "Material innovation",
    "Sustainable materials",
    "Structural analysis",
    "Construction materials",
    "Aerodynamic materials"
  ],
  "Plateforme": [
    "Digital platform",
```

```
    "Mobility platform",
    "Service integration",
    "Interoperability",
    "Platform economy",
    "Platform-as-a-Service",
    "Booking system",
    "Platform governance",
    "User interface",
    "Multiservice access",
    "API integration",
    "Platform architecture",
    "Data platform",
    "Platform deployment",
    "Cloud-based platform"
  ],
  "Rural": [
    "Rural area",
    "Rural mobility",
    "Low-density",
    "Rural development",
    "Agricultural transport",
    "Public transport accessibility",
    "Territorial equity",
    "Remote access",
    "Rural infrastructure",
    "Service availability",
    "Mobility gap",
    "Rural planning",
    "Digital divide",
    "Population dispersion",
    "Rural logistics"
  ],
  "Transfrontalier": [
    "Cross-border",
    "Border mobility",
    "Transnational",
    "Customs control",
    "International corridor",
    "Interoperability",
    "Schengen",
    "Cross-border infrastructure",
    "Binational cooperation",
    "Border regions",
    "International transport",
    "Transboundary planning",
    "EU regulation",
```

```
    "Border logistics",
    "Customs procedures"
  ],
  "Urbain": [
    "Urban mobility",
    "City planning",
    "Public transport",
    "Urban infrastructure",
    "Smart city",
    "Urban density",
    "Traffic management",
    "Urban sprawl",
    "Sustainable city",
    "Last mile",
    "Urban logistics",
    "Urban transport network",
    "Urban development",
    "Congestion",
    "Urban environment"
  ]
}
```


- **Scientific Themes**:


```
{
    "Sciences juridiques, Règlementations et Assurances": [],
    "Matériaux, Aérodynamique, Transition écologique, Énergétique et Mobilités durables": [
    "Sciences économique, Évaluation des politiques publiques, Modèles économiques": [],
    "Marketing, Durabilité, Chaine d'approvisionnement, Logistiques, Inter & Multimodalité":
    "Sciences cognitives, Interfaces H/M": [],
    "Géographie, Territoires et Sociologie": [],
    "Capteurs et Traitement du signal": [],
    "Base et Traitement de données, IA": [],
    "Communications, Infrastructures et Connectivité": [],
    "Sécurité, Cybersécurité, Résilience, Sureté, Fonctionnement et Interopérabilité des sys
}
```


- **Axes/ Leverage for action**:


```
{
  "Accompagner le développement des systèmes de transports décarbonés et sûrs": [],
  "Peser sur les choix de modes de transport des voyageurs": [],
  "Favoriser le report modal des marchandises vers le fer et le maritime et améliorer la log
}
```


- **Types of mobility**:


```
{
  "Aérien": [],
```

```
  "Ferroviaire": [],
  "Fluvial/Maritime": [],
  "Routier": [],
  "Transport par cables": []
}
```

You can see here some keywords on the **themes**. There are keywords also for the others. It helps the embedding of the *llm* model used to construct a *labellised dataset*. There are keywords only for that reason.

A publication could have:

- a *unique* **theme**. There is at least one *theme* found.
- *various* **scientific themes**. There is at least one found.
- a *unique* **axe/ leverage for action**. There could be none.
- *various* **mobility types**. There could be none.

The relevant metadatas for classification using Crossref

Here is a list of what it is possible to retrieve on a research paper using the *Crossref API*:

- The **abstract**: *Crossref* imports abstracts from ***JATS-formatted XML***, and sometimes the abstract is not found. I assume it is because of the ***paywalls***. There are multiple reasons. For instance, the authors are not every time writing it. Furthermore, the abstract is sometimes found in the **title** section, I don't even know how it is possible. Besides, the abstracts are not always written in *english*. It is not a problem if the classification model is based on *NLP*, but it will be a big problem if the model is based on *TF IDF* because this is a word comparison model. Finally, *Crossref* is very bad at finding *abstract*. It is hard to find them, is it because of *Habanero*, the *Crossref* client that I use? I don't know. Anyway, it justifies the use of another *API* such as *Semantic Scholar* for instance. This is **sometimes relevant for classification, sometimes not**, because of the language used.
- The **title**: It is always possible to find a *title* on a research paper. Same issue as the abstact, it is not always written in *english*, it's sad. It is **relevant for the classification**.
- The **DOI** of course, which is a sequence of "random" numbers/ letters. It is **not relevant for the classification**.
- The **authors**, **not relevant for the classification**.
- The **container-title** which is a list of random stuff related to the thing that contains the publication. It is not fully clear, ***this issue*** says. Anyway, it sometimes has the title of the book, or the journal. It

could be relevant for the classification. For instance, for this DOI = `10.51926/iste.9180.ch6`, the *container-title* is `"Contrôle et gestion des systèmes de transport intelligents coopératifs"`. This one is **relevant for classification**.

- The **issn-type** which can be `print` or `electronic`. For a long time I thought it was a keyword given by the author. Actually not! It just says if the publication has been released in a real book or just on a web site. So this is **not relevant for classification**.
- Finally, the *graal*. The **references**! A publication is based on other *research papers*, it cites other *research papers*. These *research papers* also have a *title*, an *abstract*, etc.. This is **relevant for the classification** because it is now possible to **recursively apply the classification model** to *references*.

So, the text to classify could be the concatenation of all those elements.

First step: create a set of labellised publications

The work of the former intern shows that the precision of the classification, for models, is bad when the model is not trained.

There are two things to be careful of:

- What is the text to classify? Is it just the *abstract*, or the *abstract* and the *title*? A concatenation of more things?
- How many *labellised publications* are needed to get good classification results?

The former work

To train the model, I need a set of *labellised pubications*. The former intern started to do that, but only on the abstract, and for the **scientific themes**. Here are some lines of the former dataset:


```
[
    {
        "abstract":"Shipping contributes roughly 2.8 % of global anthropogenic...",
        "themes":
        [
        "Matériaux, Aérodynamique, Transition écologique, Énergétique et Mobilités durables'
        "Marketing, Durabilité, Chaine d'approvisionnement, Logistiques, Inter & Multimodali
        ]
    },
    {
        "abstract":"We analyze several scenarios of transport reforms for the city of Bordea
        "themes":
```

```
        [
        "Sciences économique, Évaluation des politiques publiques, Modèles économiques"
        ]
    },
    {
        "abstract":"Cooperative intelligent transportation systems are now being widely...",
        "themes":
        [
        "Communications, Infrastructures et Connectivité", "Base et Traitement de données,
        ]
    }
]
```


Here are the results:

Modèle	Exact Match	Précision	Recall	F1-Score	Accuracy
SpaCy (taux 0.75)	21/128	25.42%	35.29%	29.56%	77.66%
SpaCy (0.75) + Prétraitement	1/128	17.90%	67.06%	28.25%	54.77%
SpaCy (taux 0.80)	20/128	29.47%	16.47%	21.13%	83.67%
SpaCy (0.80) + Prétraitement	9/128	21.13%	17.65%	19.23%	80.31%
Modèle Simple	23/128	43.93%	55.29%	48.96%	84.69%
Simple + Prétraitement	23/128	45.60%	48.82%	47.16%	85.47%
Simple + Prétraitement + Synonymes	14/128	36.58%	55.29%	44.03%	81.33%
TF-IDF (0.10) + Prétraitement	10/128	28.50%	69.41%	40.41%	72.81%
TF-IDF (0.15) + Prétraitement	12/128	32.49%	52.94%	40.27%	79.14%
TF-IDF (0.20) + Prétraitement	16/128	35.23%	40.00%	37.47%	82.27%
TF-IDF (0.25) + Prétraitement	20/128	41.22%	31.76%	35.88%	84.92%
TF-IDF (0.30) + Prétraitement	14/128	44.68%	24.71%	31.82%	85.94%
Combiné (TF-IDF + embedding 0.40) + Prétraitement	11/128	23.83%	77.65%	36.46%	64.0
Combiné (TF-IDF + embedding 0.45) + Prétraitement	25/128	40.09%	50.00%	44.50%	83.4
Combiné (TF-IDF + embedding 0.50) + Prétraitement	19/128	57.75%	24.12%	34.02%	87.5

So yeah, the **F1-score** never goes above *50%*. Even with the best *word normalization*, and *embedding*, the models are struggling.

In fact, the former intern could not train the models because of the lack of *labellised data*. For these results, there were only **131 labellised publications**.

My version

I want to train my model. It needs *labellised data*. I must do **unsupervised learning** to give it to it.

Just let's remember that we live at a fantastic time. I just used a *llm/ pretrained model*.

The *llm* needs to be accessible locally. I started to search on ***Ollama***. I found that models based on *BERT* are better than the others to classify a text.

Then, I tried to find a list of pretained models based on ***BERT*** and their precisions ***right here***. It appeared that the model ***all-MiniLM-L6-v2 from Hugging face*** is just the fastest one, and the more compact one, with a good average performance of *58.80%*. The documentation was also good, it recommended to use ***sentence-transformers***.

I gave almost everything of what *Crossref* could find on the publication.

Here is an example of a text to classify using the following format `[title][container-title][reference1-article][reference1-journal]abstract`:

```
[ Context-Aware Broadcast in Duty-Cycled Wireless Sensor Networks ]
[ Sensor Technology ]
[ An energy-efficient MAC protocol for wireless sensor networks. ]
[ Proceedings of the IEEE ]
As the energy efficiency remains a key issue in wireless sensor networks,
duty-cycled mechanisms acquired much interest due to their ability to reduce
energy consumption by allowing sensor nodes to switch to the sleeping state
whenever possible. The challenging task is to authorize a sensor node to
adopt a duty-cycle mode without inflicting any negative impact on the
performance of the network. A context-aware paradigm allows sensors to adapt
their functional behavior according to the context in order to enhance
network performances. In this context, the authors propose an enhanced
version the Efficient Context-Aware Multi-hop Broadcasting (E-ECAB) protocol,
which combines the advantages of context awareness by considering a multi
criteria and duty-cycle technique in order to optimize resources usage and
satisfy the application requirements. Simulation results show that E-ECAB
achieves a significant improvement in term of throughput and end-to-end delay
without sacrificing energy efficiency.
```

A reference is a publication that has been cited by the current publication.

Et Voila!

```
(.venv)  backend  >>  python tests.py
Number of abstracts: 128
Exact Match Count: 57

Global Precision: 58.45%
Global Recall: 71.18%
Global F1-Score: 64.19%
Global Accuracy: 89.45%
```

I found that, the more you give to it, the better the precision will be, even if the text language is not in english.

This way, I can now try to construct the *labellized dataset*. However, it will require a *human check* because of the precision. Anyway, the precision is already very high, it is hard to get better results without training the model.

Second step: train the TF IDF model

First of all, why *TF IDF*?

Claude Feldges, a data scientist, published ***right here*** a **performance comparison** on text classification between *TF IDF*, *LSTM* and *BERT*. It appears that *TF IDF* is so good with like *97%* precision on his dataset. Like he said, `"this classification is a simple problem"`, and `"more complex models does not improve accuracy, but costs much more time"`, so yeah, I will `"keep it simple"`.

I assume that I just need to follow his masterclass to construct a strong *TF IDF* based model. He encourages us to use *logistics regression* as the *classifier*. I found this work ***right here*** that is using *Native Bayes* as a classifier.

I would also like, on my own, to wrap it with *Similarity Cosine* just to experiment things.

It could also be necessary to normalize the data. I found this work that is doing it well ***right here***.

EOF